

Linux for Computational Chemistry

Lecture 2: Text Processing, Redirection and Pipes

Emilio Martínez Núñez

Master in Theoretical and Computational Chemistry



Outline

Recap and streams

Redirection and Pipes

grep: finding patterns

sed: stream editing

awk: columns and simple logic

Putting it together

Recap and streams

Quick recap of Lecture 1

- Navigation: `pwd`, `ls`, `cd`
- File operations: `cp`, `mv`, `rm`, `mkdir`
- Permissions: `chmod`
- Today: turning raw calculation output into the numbers you actually need

The three standard streams

Every command has three data channels:

- **stdin** (0) — standard input, normally the keyboard
- **stdout** (1) — standard output, normally the screen
- **stderr** (2) — standard error, also normally the screen, but a *separate* channel

Note

Gaussian and most quantum chemistry codes write results to stdout and warnings/crashes to stderr. Keeping them separate lets you filter errors from real output.

Redirection and Pipes

Redirecting output

```
g16 < opt.com > opt.log # stdin from file, stdout to file  
g16 < opt.com > opt.log 2> opt.err # stderr to a separate file  
g16 < opt.com &> opt.full # both stdout and stderr together  
g16 < opt.com >> opt.log # append instead of overwrite
```

Tip

> overwrites silently. If you rerun a job and forget this, you lose the previous log. Use » when in doubt, or rename first.

Pipes: chaining commands

```
command1 | command2
```

- The stdout of `command1` becomes the stdin of `command2`
- This is how small, single-purpose tools are combined into powerful pipelines

```
cat opt.log | grep "SCF Done" | wc -l
```

Exercise 2.1

Assume you have a script `fake_gaussian.sh` that prints some lines to `stdout` and one line `"WARNING: low memory"` to `stderr` (ask your instructor for the script, or write one with `echo "... " >&2`).

1. Run it so `stdout` goes to `run.log` and `stderr` goes to `run.err`.
2. Run it again so *everything* goes into a single file `run_full.log`.
3. Confirm the warning appears only in `run.err`, not `run.log`.

Solution

```
./fake_gaussian.sh > run.log 2> run.err  
./fake_gaussian.sh &> run_full.log  
grep WARNING run.log # nothing found  
grep WARNING run.err # the warning is here
```

grep: finding patterns

grep: your most-used tool

```
grep "SCF Done" opt.log # lines containing the text
grep -i "error" opt.log # case-insensitive
grep -v "SCF Done" opt.log # invert: lines NOT matching
grep -c "SCF Done" opt.log # count matching lines
grep -n "SCF Done" opt.log # show line numbers
grep -r "Normal termination" . # search recursively in a directory
grep -l "Error termination" *.log # just list matching filenames
```

A touch of regular expressions

```
grep "^ SCF Done" opt.log # ^ anchors to start of line
grep "Hartree$" opt.log # $ anchors to end of line
grep "E(RB3LYP)" opt.log # literal parentheses need escaping: \(
grep -E "[0-9]+\.[0-9]+" opt.log # -E enables extended regex: digits.digits
```

Note

You do not need to master regular expressions today — just recognize `^`, `$`, `.`, and `*` when you see them.

Exercise 2.2

Given a set of Gaussian log files `conf1.log ... conf5.log` in the current directory:

1. Count how many of them contain the string "Normal termination".
2. List the names of the files that instead contain "Error termination".
3. Print, with line numbers, every line in `conf1.log` that contains "SCF Done".

Solution

```
grep -l "Normal termination" *.log | wc -l  
grep -l "Error termination" *.log  
grep -n "SCF Done" conf1.log
```

sed: stream editing

sed: search and replace

```
sed 's/old/new/' file.txt # replace first match per line, print result
sed 's/old/new/g' file.txt # replace ALL matches per line
sed -i 's/old/new/g' file.txt # edit the file IN PLACE
sed -i.bak 's/old/new/g' file.txt # in place, keeping a .bak backup
sed '/pattern/d' file.txt # delete lines matching pattern
```

Tip

Always test a sed command *without* `-i` first to see the result printed to screen, before overwriting the file.

Exercise 2.3

You have an `.xyz` file where element symbols were exported in lowercase (`c`, `h`, `o`) instead of the standard `C`, `H`, `O`, and it also contains stray blank comment lines starting with `#` that must be removed.

1. Write a `sed` command (tested without `-i` first) that replaces every standalone `c` field with `C` (careful: think about why replacing the raw letter everywhere would be dangerous inside a coordinates file).
2. Write a `sed` command to delete every line starting with `#`.
3. Combine both edits in place, keeping a `.bak` backup.

Solution

```
sed 's/^c /C /' coords.xyz  
sed '/^#/d' coords.xyz  
sed -i.bak -e 's/^c /C /' -e '/^#/d' coords.xyz
```

Note

Anchoring to the start of the line (^c) avoids accidentally replacing a "c" that is part of a longer element symbol like "Ca" or "Cl".

awk: columns and simple logic

awk: thinking in columns

```
awk '{print $1}' coords.xyz # print the first whitespace-separated field
awk '{print $1, $4}' coords.xyz # print field 1 and field 4
awk 'NF==4 {print}' coords.xyz # only lines with exactly 4 fields
awk 'NR>2 {print}' coords.xyz # skip the first two lines (header)
awk -F"," summary.csv '{print $2}' # use comma as the field separator
```

- \$1, \$2... — fields; \$0 — the whole line
- NF — number of fields; NR — current line/record number

```
awk '{sum += $2} END {print sum}' summary.csv  
awk '{sum += $2; n++;} END {print sum/n}' summary.csv  
awk '$2 < -155.0 {print $1}' summary.csv
```

Note

This turns awk into a lightweight calculator over tabular data — extremely useful for a quick sanity check before opening Python or Excel.

Exercise 2.4 — Extracting and averaging energies

Exercise 2.4

You have a file `summary.csv` with two comma-separated columns: conformer name and final SCF energy in Hartree.

1. Print only the energy column.
2. Compute the average energy over all conformers.
3. Print the names of conformers with an energy below -155.0 Hartree.

Solution

```
awk -F',' '{print $2}' summary.csv  
awk -F',' '{sum+=$2; n++} END {print sum/n}' summary.csv  
awk -F',' '$2 < -155.0 {print $1}' summary.csv
```

Putting it together

Combining pipes: a real pipeline

```
grep "SCF Done" *.log \  
| awk '{print FILENAME, $5}' \  
| sort -k2,2n
```

- `grep` finds the energy lines across all log files (keeping the filename with multi-file `grep`)
- `awk` extracts just the filename and energy value
- `sort -k2,2n` sorts numerically by the second column
- Also useful: `sort -u` (unique), `uniq -c` (count repeats after sorting)

Exercise 2.5

For a directory containing `conf1.log` ... `conf10.log` Gaussian output files:

1. Build a single pipeline that extracts, from every file, the last "SCF Done" energy value, together with the filename.
2. Sort the result from lowest to highest energy.
3. Redirect the final sorted table into a file `energies_sorted.txt`.

Solution

```
grep "SCF Done" *.log \  
| awk '{print FILENAME, $5}' \  
| sort -k2,2n \  
> energies_sorted.txt
```

- Redirection (`>`, `»`, `2>`, `&>`) separates and controls where output goes
- `grep` finds lines, `sed` edits text, `awk` works with columns and numbers
- Pipes (`|`) chain all of these into one-line pipelines
- This is precisely how you turn 50 raw log files into one clean results table

Exercise 2.6 (Homework)

For a directory of Gaussian log files:

1. Build a pipeline that reports, for each file, whether it ended with "Normal termination" or "Error termination".
2. Produce a single summary line: "X of Y jobs finished normally".
3. Save both results to a file `status_report.txt`.

We will revisit this exact problem next class and turn it into a reusable script.